NOMBRE Y APELLIDOS:

FECHA: 16/03/2026

EXP.: SIMULACRO

Python y Django (IFCT095PO)

Duración: 60 minutos | Puntuación total: 10 puntos

Mínimo:

- 1,5/4 en test
- 1,5/6 en práctico

Instrucciones:

Clonad el repositorio:

```
git clone https://github.com/elMestreAcademy/2026-IFCT095PO-GR1-prueba
```

Responded sobre el fichero `RESPUESTA.md`

Test (10 preguntas, 1 punto c/u)

1. ¿Cuál es la salida de `print(3 * "ab")`?

- a) `3ab`
- b) `ababab`
- c) `ab3`
- d) Error de tipo

2. ¿Qué comando crea una nueva aplicación llamada `blog` dentro de un proyecto Django existente?

- a) `django-admin startproject blog`
- b) `python manage.py startapp blog`
- c) `python manage.py newapp blog`
- d) `django-admin createapp blog`

3. ¿Cuál es el resultado de `len({"a": 1, "b": 2, "a": 3})`?

- a) `3`
- b) `2`
- c) `6`
- d) Error: clave duplicada

4. ¿Qué comando genera los ficheros de migración tras modificar un modelo?

- a) `python manage.py migrate`
- b) `python manage.py makemigrations`
- c) `python manage.py sqlmigrate`
- d) `python manage.py dbsync`

5. ¿Cómo se instancia un formulario Django con los datos recibidos de una petición POST?

- a) `MiForm(data=request.body)`
- b) `MiForm(request.POST)`
- c) `MiForm.from_post(request)`
- d) `MiForm(fields=request.POST)`

6. ¿Qué método devuelve el número total de registros de un QuerySet?

- a) `queryset.size()`
- b) `queryset.count()`
- c) `queryset.length()`
- d) `len(queryset)`

7. ¿Qué atributo de `ModelAdmin` define las columnas visibles en el listado?

- a) `fields`
- b) `list_display`
- c) `columns`
- d) `display_fields`

8. ¿Qué middleware Django gestiona la protección contra ataques CSRF?

- a) `django.middleware.security.SecurityMiddleware`
- b) `django.contrib.auth.middleware.CSRFMiddleware`
- c) `django.middleware.common.CsrfMiddleware`
- d) `django.middleware.csrf.CsrfViewMiddleware`

Pregunta	1	2	3	4	5	6	7	8
Respuesta								

Supuesto práctico

Ejercicio 1: (2 puntos)

Defina dos modelos Django: `Autor` y `Libro`.

Autor:

- `nombre`: texto, máximo 150 caracteres
- `email`: email, único
- `activo`: booleano, por defecto `True`

Libro:

- `titulo`: texto, máximo 300 caracteres
- `autor`: clave foránea a `Autor` (si se borra el autor, el libro mantiene el campo como `NULL`)
- `precio`: decimal, 8 dígitos totales, 2 decimales
- `publicado`: fecha, puede estar vacía/nula
- `autores_secundarios`: relación muchos-a-muchos con `Autor` (puede estar vacía)

Añada al modelo `Libro` un método `precio_con_iva(self)` que devuelva el precio multiplicado por 1.21 redondeado a 2 decimales.

Ejercicio 2:(2 puntos)

Escriba:

a) Una vista basada en función llamada `libros_por_autor` que:

- Reciba `request` y un parámetro `autor_id`
- Obtenga el objeto `Autor` o devuelva un 404 si no existe
- Filtre los libros de ese autor que tengan fecha de publicación (`publicado` no nulo)
- Pase al template `libros/por_autor.html` el autor y los libros

b) La entrada en `urls.py` que mapee la ruta `autor/<int:autor_id>/libros/` a esa vista con el nombre `libros-por-autor`.

c) El template `libros/por_autor.html` que:

- Extienda de `base.html`
- Muestre como título `<h1>` el nombre del autor

- Liste los libros en una lista `<ul>` mostrando título y precio con IVA (usando el método del modelo)
- Si no hay libros, muestre "Este autor no tiene libros publicados."

Ejercicio 3: (2 puntos)

a) Cree un `ModelForm` llamado `LibroForm` para `Libro` con campos: `titulo`, `precio`, `publicado`, `autor`.

b) Implemente una validación en el método `clean()` del formulario que verifique: si el campo `publicado` tiene fecha, el `precio` no puede ser 0. Si se incumple, lance un `ValidationError` con el mensaje "Un libro publicado debe tener precio mayor que 0."

c) Escriba una vista `crear_libro` que:

- En GET: muestre el formulario vacío
- En POST: valide el formulario; si es válido lo guarde y redirija a `libros-por-autor` del autor correspondiente; si no es válido, vuelva a mostrar el formulario con los errores
- Use el template `libros/crear.html`